Lista de Exercícios

1. Escreva um programa que leia um número N e calcule a soma de todos os números de 1 a N usando um laço for.

2. Faça um programa que leia as idades de 10 pessoas e calcule a média. Use for.

3. Crie um programa que leia um número e determine se ele é primo. Use um laço for e uma flag.

4. Refaça o exercício da calculadora com menu, mas agora o programa deve continuar executando até que o usuário escolha a opção "Sair". Use do-while.

5. Imprima os números de 10 a 1 usando while e depois usando for.

6. Faça um programa que solicite um número positivo. Se o usuário digitar um número negativo ou zero, o programa deve pedir novamente (usando while). Ao final, mostre o número válido.

7. Imprima as potências de 2 (2^0, 2^1, 2^2, ...) até que o valor seja menor que 1000. Use for.

8. Crie um programa que desenhe um retângulo de asteriscos com altura e largura fornecidas pelo usuário. Use laços aninhados (for).

9. Crie um programa que desenhe um triângulo retângulo de asteriscos, onde a base é fornecida pelo usuário. Use laços aninhados.

10. Refaça o programa da sequência de Fibonacci que imprime os primeiros N termos, mas agora use while ou do-while. Compare com a versão anterior.

11. Crie um programa que preencha um array de 10 posições com números aleatórios (use rand()) e, em seguida, imprima todos os elementos.

12. Faça um programa que leia dois arrays de 5 elementos cada e gere um terceiro array que seja a soma elemento a elemento.

13. Escreva um programa que leia um array de 10 números e, em seguida, leia um número-chave. O programa deve procurar a chave no array e informar se foi encontrada e em qual posição (índice).

14. Crie um programa que leia 5 números, armazene-os em um array e, usando um algoritmo de ordenação simples (Bubble Sort), ordene e exiba o array.

15. Faça um programa que leia uma matriz 2x3 e calcule a soma de cada linha, armazenando os resultados em um array de 2 posições.

16. Crie um programa que leia uma matriz 3x3 e calcule a soma dos elementos da diagonal principal.

17. Leia os nomes de 5 cidades e os armazene em um array de strings (char nomes[5][50]). Em seguida, imprima a lista de cidades.

18. Escreva um programa que copie o conteúdo de um array para outro, sem usar funções de biblioteca.

19. Leia um array de 15 números (0 a 9). Conte quantas vezes cada número aparece e imprima a frequência.

20. Crie um programa que leia um número N, aloque dinamicamente um array de N inteiros (usando malloc), preencha-o com valores e o imprima.

21. Crie um programa que escreva a string "Primeira linha de um arquivo." em um arquivo chamado teste.txt.

22. Faça um programa que leia o conteúdo de teste.txt e o exiba no console.

23. Crie um programa que peça ao usuário para digitar 5 nomes e os grave no arquivo nomes.txt, um por linha.

24. Leia o arquivo nomes.txt e exiba na tela quantos nomes têm mais de 5 caracteres.

25. Modifique o programa 3 para que ele anexe novos nomes ao final do arquivo, sem sobrescrever.

26. Crie um programa que leia o nome, idade e salário de um funcionário e grave esses dados em um arquivo funcionario.csv no formato "Nome,Idade,Salario".

27. Leia o arquivo funcionario.csv, processe cada linha e exiba as informações formatadas.

28. Crie um arquivo de texto com vários números inteiros (um por linha). Crie um programa que leia esse arquivo, some todos os números e exiba o resultado.

29. Escreva um programa que leia o conteúdo de um arquivo de origem e copie para um arquivo de destino.

30. Crie um programa com menu que permita: 1) Inserir dados no arquivo, 2) Exibir todos os dados, 3) Sair. Use um loop do-while e funções para organizar.

31. Crie uma função soma(int a, int b) que retorna a soma. No main, leia dois números e chame a função.

32. Crie uma função exibeMensagem() que apenas imprima "Bem-vindo ao sistema!" no terminal.

33. Crie uma função que recebe um inteiro N e retorna o fatorial de N (N!). Use um laço for na função.

34. Crie a mesma função, mas implementada de forma recursiva. Compare os códigos.

35. Crie uma função validarCPF(char cpf[]) que retorne 1 se o CPF for válido (considere apenas o formato, não os dígitos verificadores) e 0 caso contrário. (Dica: use strlen para contar).

36. Crie uma struct Aluno com nome e nota. Crie uma função mediaTurma(struct Aluno alunos[], int n) que calcula a média da turma.

37. Crie uma função dobrar(int *x) que dobra o valor da variável passada.

38. Separe a função soma e dobrar em um arquivo util.c e um header util.h. Use no main.c.

39. Crie uma função ordenar(int arr[], int n) que ordena um array usando Bubble Sort.

40. Função de Busca: Crie uma função buscar(int arr[], int n, int chave) que retorna o índice da chave no array, ou -1 se não encontrado.

41. Conceito e Sintaxe: Explique o que é um ponteiro e qual a diferença entre *p e &variavel.

42. Crie um programa que declare um inteiro x=10, um ponteiro p que aponte para x. Imprima o valor de x usando o ponteiro.

43. Usando o ponteiro p do exercício anterior, altere o valor de x para 20 e imprima-o.

44. Crie um array de inteiros e use um ponteiro para percorrê-lo e imprimir seus elementos.

45. Crie uma função troca(int *a, int *b) que troca os valores de dois inteiros. Use no main.

46. Crie uma struct Pessoa com nome e idade. Crie uma função inicializar(struct Pessoa *p, char nome[], int idade) que usa um ponteiro para modificar a struct.

47. Crie um array de 5 inteiros. Use um ponteiro e aritmética de ponteiros para somar todos os elementos.

48. Declare um inteiro x, um ponteiro p para x e um ponteiro q para p. Imprima o valor de x usando q.

49. Use malloc e ponteiros para criar um array de inteiros de tamanho N, preenchê-lo e liberar a memória.

50. Crie um array de ponteiros para funções que apontam para funções de operações matemáticas (soma, sub, mult, div). Use um índice para escolher e executar a operação.